

하이퍼파라미터 튜닝 & 머신러닝

CONTENTS

하이퍼파라미터 튜닝

01 하이퍼파라미터 튜닝 개념

02 튜닝 기법
& 대표적 하이퍼파라미터

03 한계점 및 실무 적용 팁

머신러닝

01 머신러닝 개념

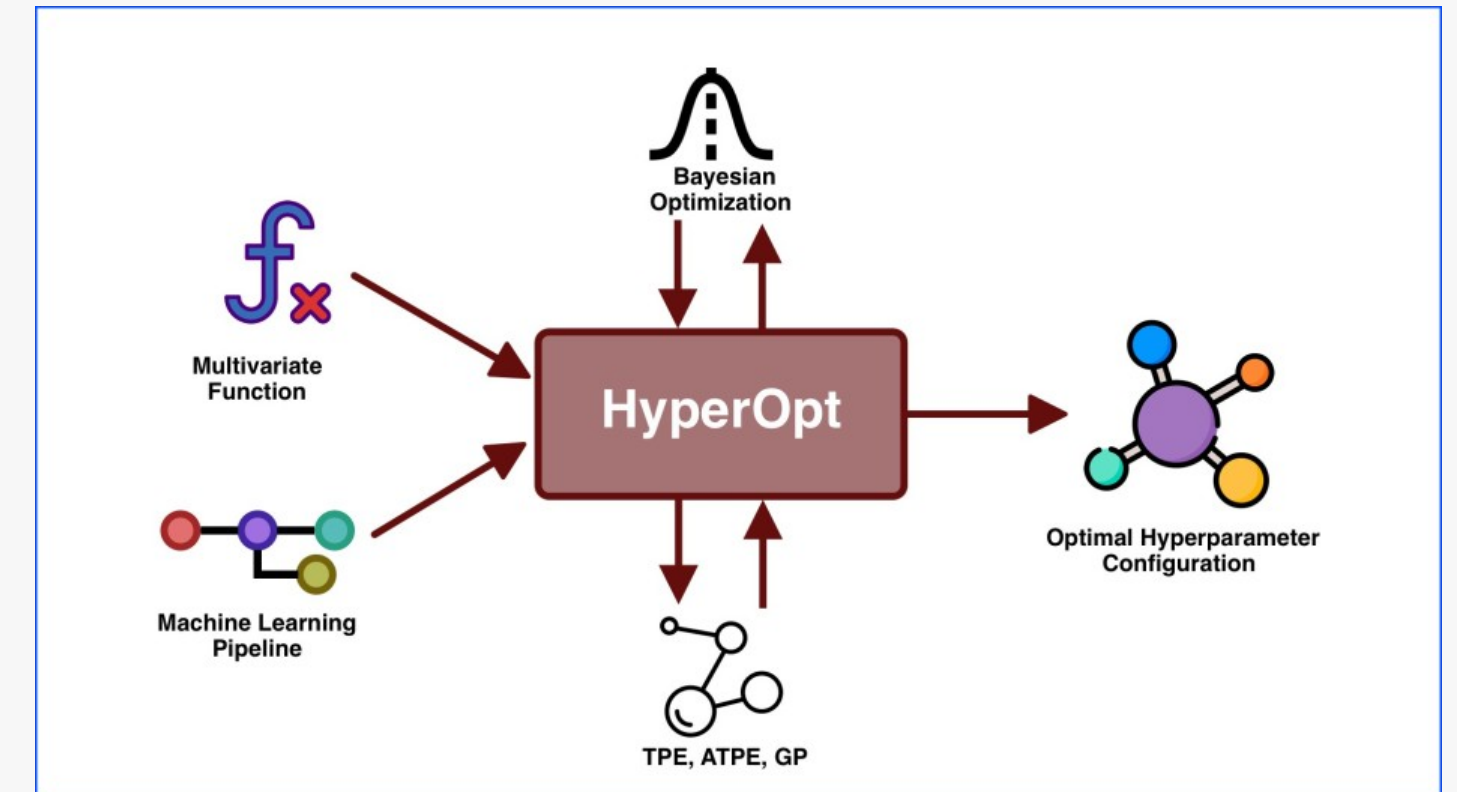
02 머신러닝 종류

03 머신러닝 알고리즘 선택

하이퍼파라미터 튜닝 개념

1. 하이퍼파라미터란

- 인공지능이나 머신러닝 모델이 데이터를 학습하기 전에 사람이 직접 설정하는 매개변수.
- 모델의 학습과 일반화 능력에 중요한 영향을 미치며, 적절한 하이퍼 파라미터 값을 찾는 것이 모델의 성능을 최적화하는데 필수적.



2. 하이퍼파라미터 튜닝

- 머신러닝 모델의 성능을 최적화 시키기 위해 모델의 최적의 하이퍼 파라미터 값을 조정하는 과정.
- 대표적으로 그리드 서치(Grid Search), 랜덤 서치(Random Search), 베이지안 최적화(Baysian Optimization)가 널리 사용된다.
- Hyperparameter Optimization 이라고 불리기도 함.

하이퍼파라미터 튜닝 개념

3. 하이퍼파라미터와 파라미터의 차이

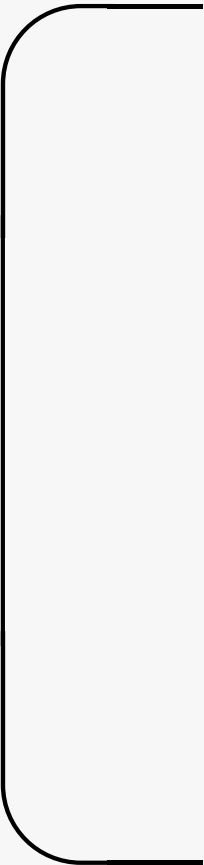
- 파라미터는 모델 내부의 존재하는 매개변수이며 학습의 대상으로 학습 알고리즘으로 자동적으로 학습.
(ex. 정규분포의 평균, 표준편차)
- 하이퍼파라미터는 경험, 데이터의 특성에 근거해 사용자가 설정하는 값. (ex. 학습률, 에포크 수)

4. 하이퍼파라미터 튜닝의 중요성

- 모델의 성능을 최적화하고 모델이 학습 데이터에만 과도하게 맞춰있는 '과적합' 문제나, 새로운 상황에 대응하지 못하는 '과소적합' 방지. 즉, 편향(bias)와 분산(Variance) 조정.

하이퍼파라미터 튜닝 기법

Automated
Hyperparameter
Selection



Manual Search

Grid search

Random Search

Bayesian Optimization

Non-Probabilistic

Evolutionary Algorithms

하이퍼파라미터 튜닝 기법 - Manual Search

매뉴얼 서치 (수동 탐색, Manual Search)

- 직관이나 경험에 기반해 직접 찾는 방법.
- 임의의 값을 대입해 결과를 살펴보고, 결과에 따라 값을 조정해가며 변화를 관찰.

장점

- 초기 베이스라인 구축에 유리
- 컴퓨팅 자원 절약
- 모델 하이퍼파라미터 민감도 파악

단점

- 높은 공수 및 시간 소모
- 최적성 미보장
- 복잡한 모델 적용 불가

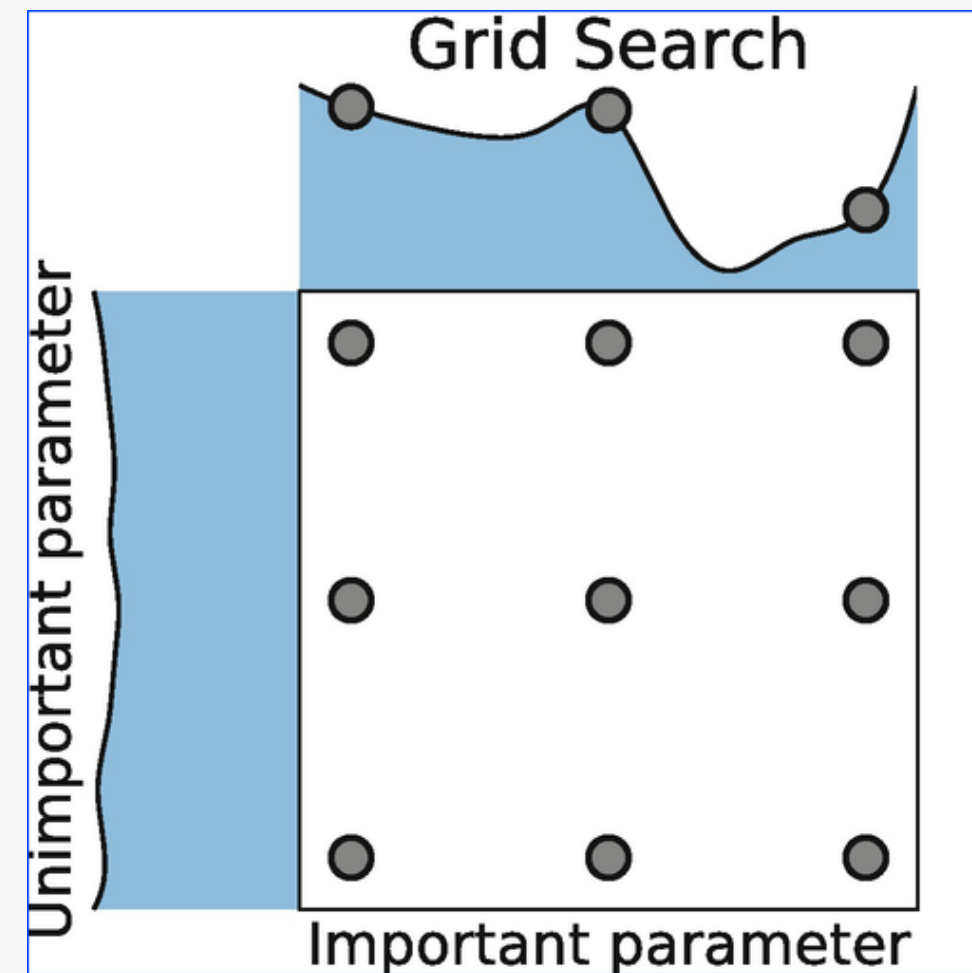
하이퍼파라미터 튜닝 기법 - Grid Search

그리드 서치 (Grid Search)

- 탐색할 하이퍼파라미터의 범위를 정해 두고 그 안에서 일정한 간격으로 값을 대입해보는 방식

특징

- 완전 탐색
- 100% 재현성
- 차원의 문제
- 우수한 병렬 처리성
- 이산적 탐색점의 한계



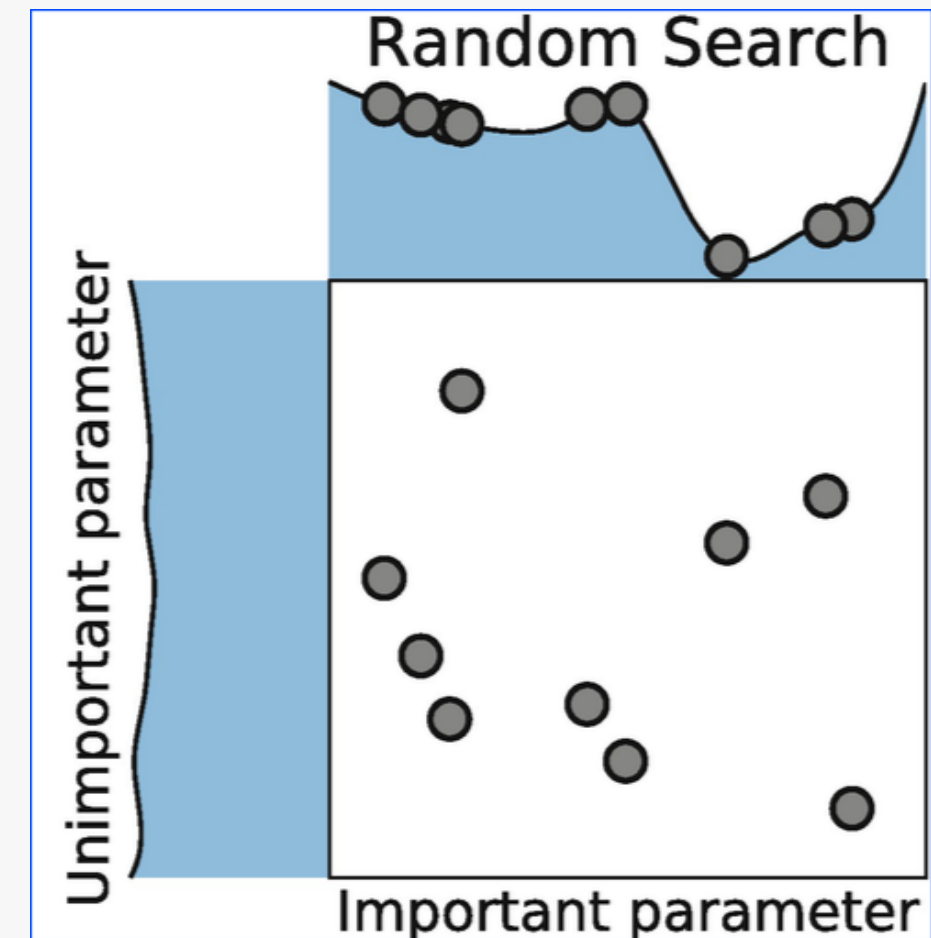
하이퍼파라미터 튜닝 기법 - Random Search

랜덤 서치 (Random Search)

- 경계 내에서 임의의 조합을 추출하여 최적의 조합을 찾는 방법
- 상대적으로 Grid Search에 비해 시간 대비 성능이 좋다.

특징

- 시간 예산 보장(n_{tier})
- 분포 기반 샘플
- 다양한 값 검증
- 실행 시점별 변동성
- 이전 결과 미참조



하이퍼파라미터 튜닝 기법 - Bayesian Optimization

베이지안 최적화 (Bayesian Optimization)

- 베이즈 정리(Bayes' theorem)를 기반으로 미지의 목적함수를 최대 또는 최소로 하는 최적해를 찾는 방법
- 과거의 평가 결과를 바탕으로 최적의 하이퍼파라미터 탐색 위치를 추천하는 순차적 모델 기반 최적화(SMBO) 기법.

베이지안 최적화의 두가지 핵심 요소

- 대체 모델
실제 하이퍼파라미터에 따른 모델의 성능(목적 함수)을 예측하는 확률적 추정 모델.
- 획득 함수
대체 모델이 출력한 평균과 불확실성 정보를 전달받아, "다음은 어느 하이퍼파라미터를 검토하는 것이 이득인가?" 를 수치화하여 다음 탐색 지점을 결정하는 함수

특징

- 최소 시도 횟수로 최적점 도달
- 고비용 모델 최적화에 특화
- 연속/이산 파라미터 통합 처리
- 탐색 자체의 오버헤드
- 초고차원 한계
- 구현 복잡도

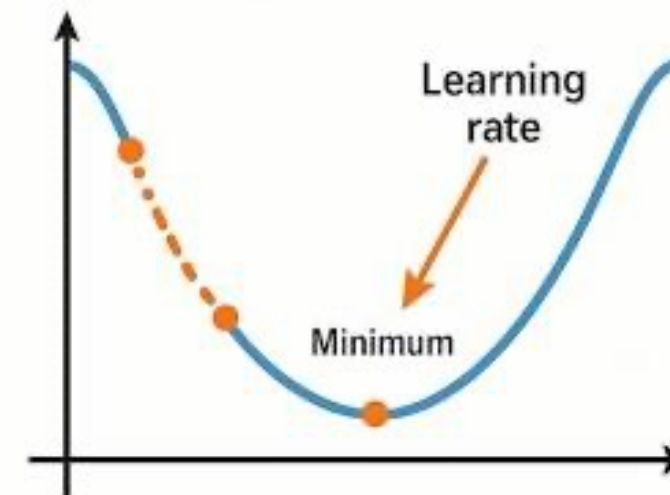
하이퍼파라미터 - 학습률

학습률 (Learning Rate)

- 경사하강법(Gradient Descent) 기반의 모델(딥러닝, XGBoost, LightGBM 등)에서 손실 함수의 최솟값을 찾아갈 때 가중치를 얼마나 크게 업데이트할지(보폭)를 결정하는 핵심 파라미터.

- 너무 큰 경우: 최솟값을 지나쳐 지나치게 발산 (Overshooting)하거나 손실 값이 진동.
- 너무 작은 경우: 학습 속도가 지나치게 느려지거나, 전역 최적점(Global Optimum)에 도달하기 전에 국소 최적점(Local Minima)에 갇힐 수 있다.

학습률(Learning Rate)과 경사 하강법(Gradient Descent)



하이퍼파라미터 - 배치 사이즈

배치 사이즈 (Batch Size)

- 딥러닝 모델이 한 번의 학습 스텝에서 처리하는 데이터 샘플 수.

적절한 배치 사이즈

항목	권장 배치 사이즈
이미지 분류 (CNN)	32 ~ 256
자연어 처리 (RNN/Transformer)	16 ~ 64
작은 데이터셋	8 ~ 32
대용량 GPU 사용	128 ~ 1024

너무 작은 배치 사이즈 (ex. 8 ~ 32)

- 장점
일반화 성능이 좋을 수 있다.
메모리 효율이 좋다.
- 단점
학습 속도가 느리다. (많은 반복 필요)
gradient의 분산이 커서 불안정할 수 있다.
BatchNorm 사용 시 통계가 불안정할 수 있다.

너무 큰 배치 사이즈 (ex. 128 ~ 1024)

- 장점
학습 속도가 빠르다.
gradient가 더 안정적.
- 단점
과적합 위험 증가.
일반화 성능이 떨어질 수 있다.
메모리 사용량이 크다.

하이퍼파라미터 - 에폭

에폭 (Epoch)

- 전체 학습 데이터셋을 모델이 처음부터 끝까지 '한 번 완벽하게 통과하며 학습을 완료한 상태'를 의미하는 단위.

전체적인 구동 예시

- 전체 데이터셋: 이미지 1,000장
 - 배치 사이즈 (Batch Size): 100장 (1회 가중치 업데이트 단위)
 - 이터레이션 (Iteration / Step): $10,000 \div 100 = 100$ 회
- 모델은 100개의 데이터를 넣고 학습하는 과정을 100번 반복 (100 Iterations)해야 비로소 1 에폭(1 Epoch)을 완성하게 된다.

에폭의 수

- 너무 적은 경우 → 과소적합
- 적절한 경우 → 최적점
- 너무 많은 경우 → 과적합

실무에서 에폭을 관리하는 기법

- 조기 종료 (Early Stopping)
- 모델 체크포인트 (Model Checkpointing)

하이퍼파라미터 튜닝의 한계 및 실무 적용 팁

주요 한계점

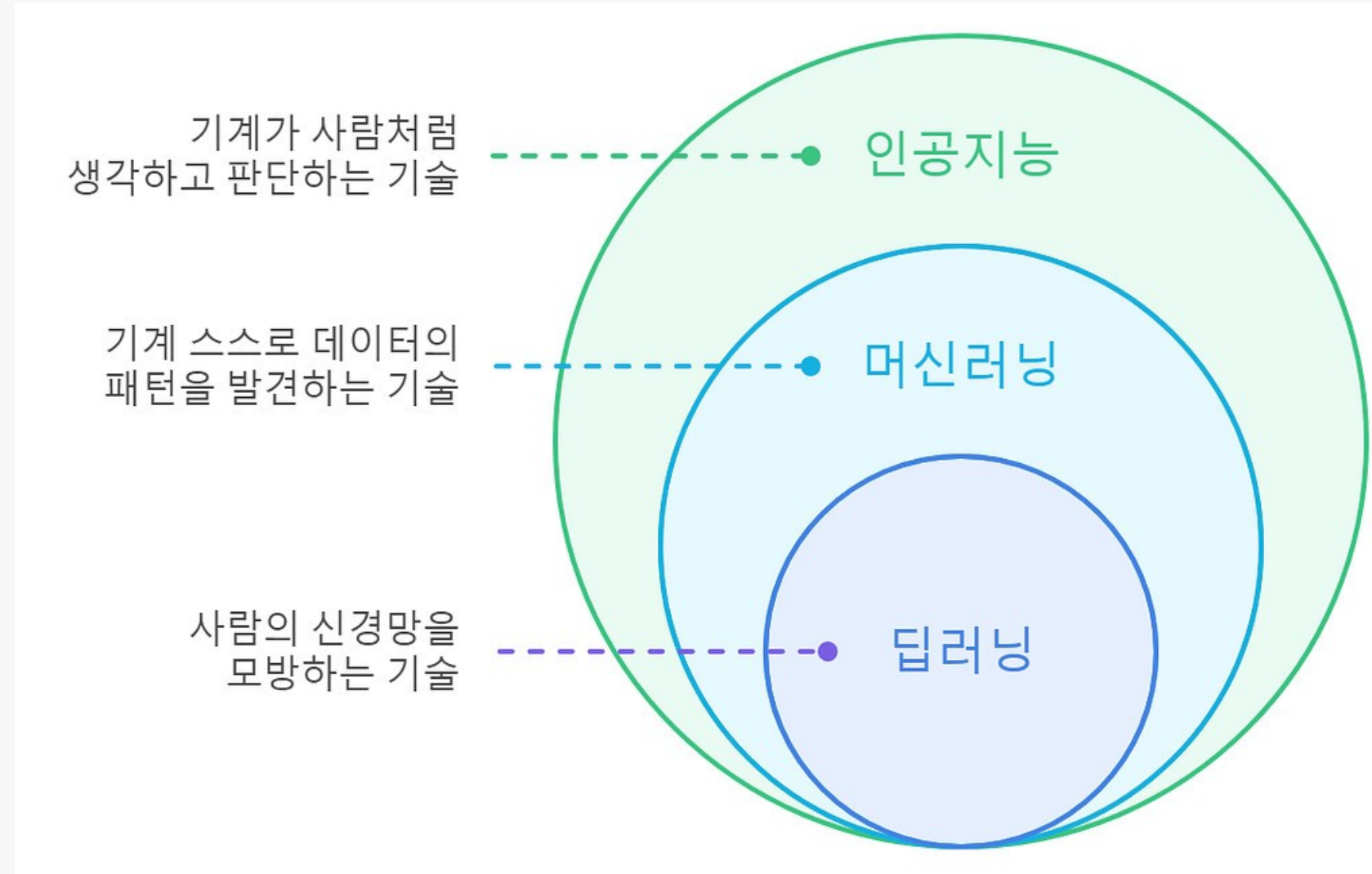
- 높은 계산 비용 (차원의 저주)
- 데이터·모델 구조의 근본 한계 극복 불가
- 과적합 위험
- 범용적인 최적값의 부재
- 탐색 공간의 폭발적인 증가

실무 적용 팁

- 베이스라인(Default) 성능부터 측정
- 핵심 파라미터 우선순위 설정
- 성긴 탐색 후 정밀 탐색
- 지수적 변수는 로그 스케일(Log Scale)로 탐색
- 자동화 도구 및 조기 종료 결합
- 교차 검증으로 검증 데이터 과적합 방지

머신러닝 개념

인공지능 ⊃ 머신러닝 ⊃ 딥러닝



머신러닝 : 사람이 직접 데이터의 특징을 가공하고 추출하여 기계에 입력

딥러닝 : 기계가 방대한 데이터 속에서 스스로 특징을 찾아내어 학습

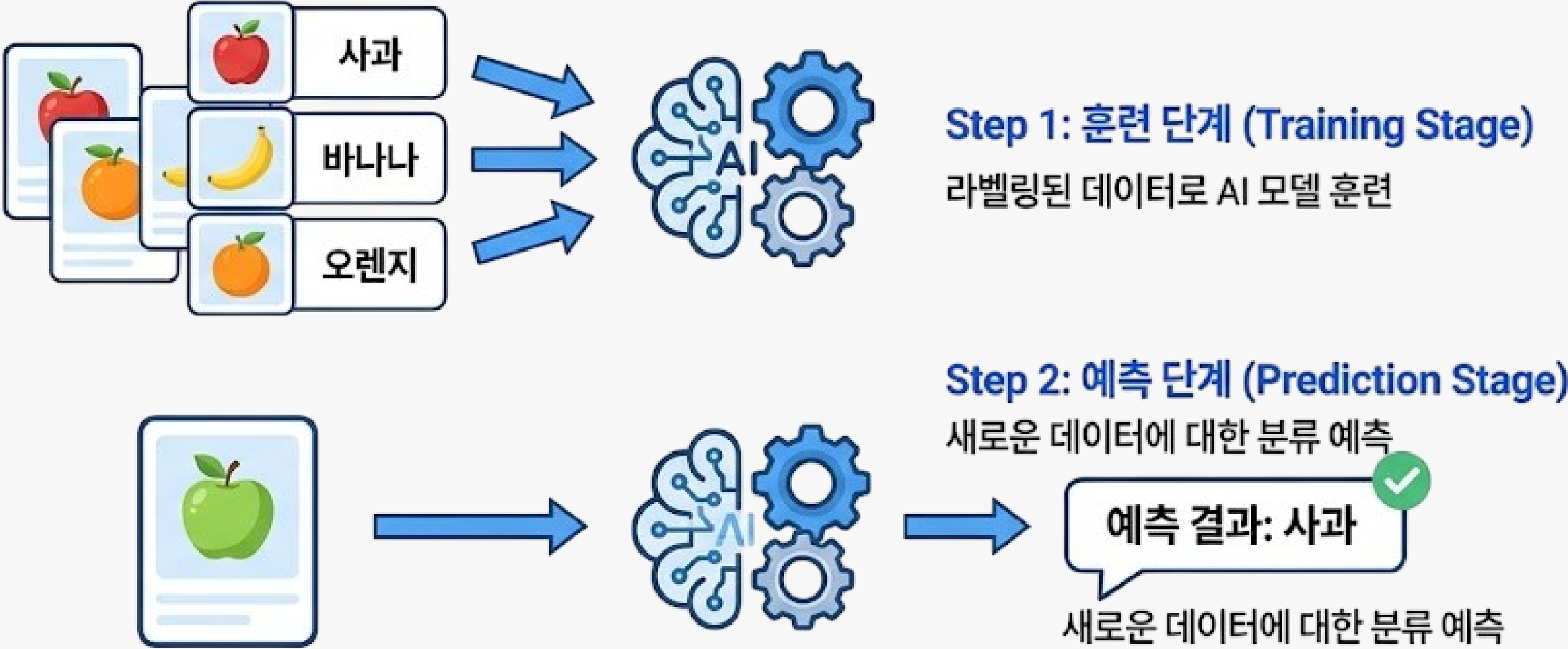
머신러닝 종류

훈련 지도 방식에 따른 분류



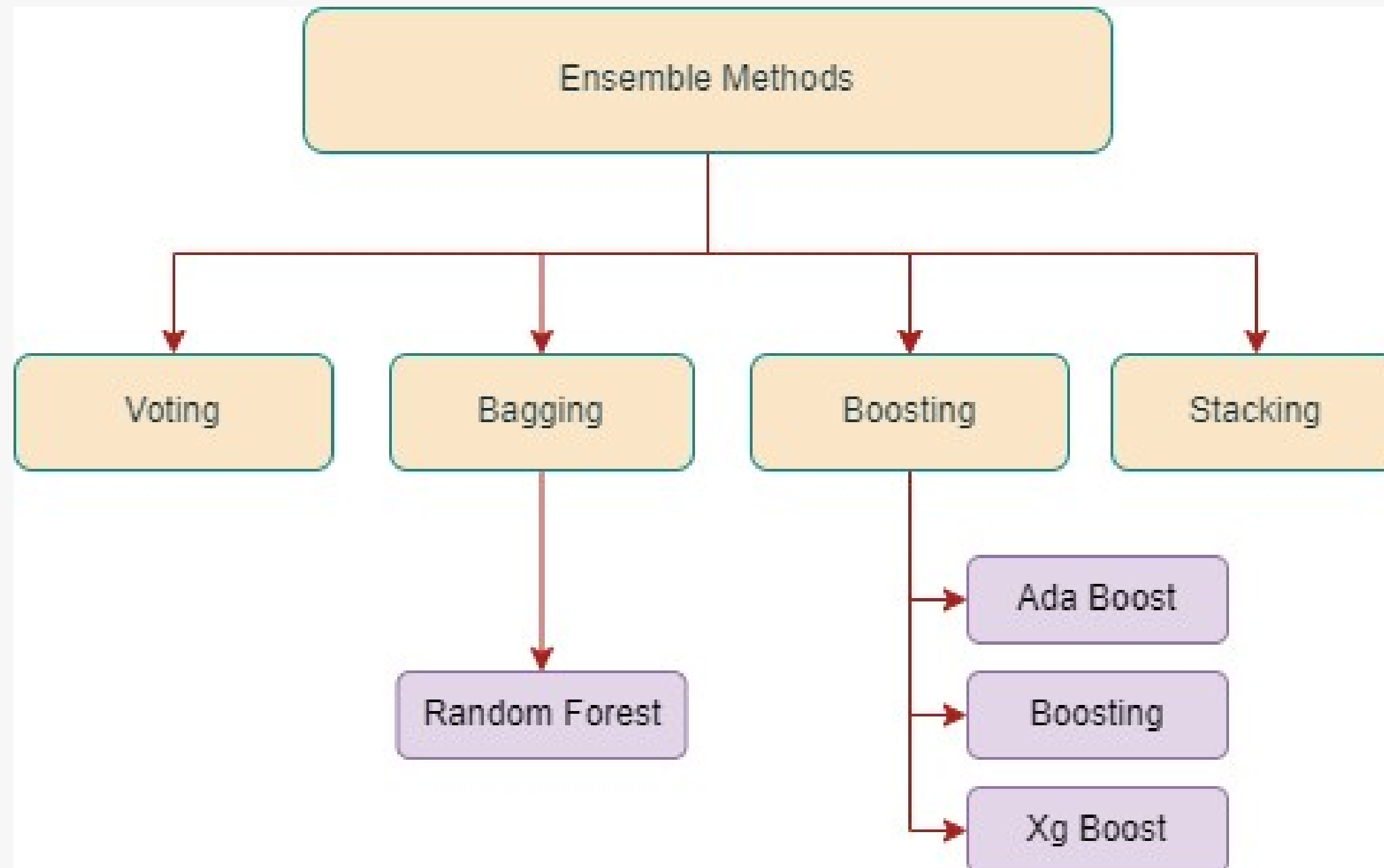
머신러닝 종류

지도학습 (Supervised Learning) : 정답이 주어진 데이터를 학습하여 새로운 데이터의 결과를 예측



머신러닝 종류 - 사전지식

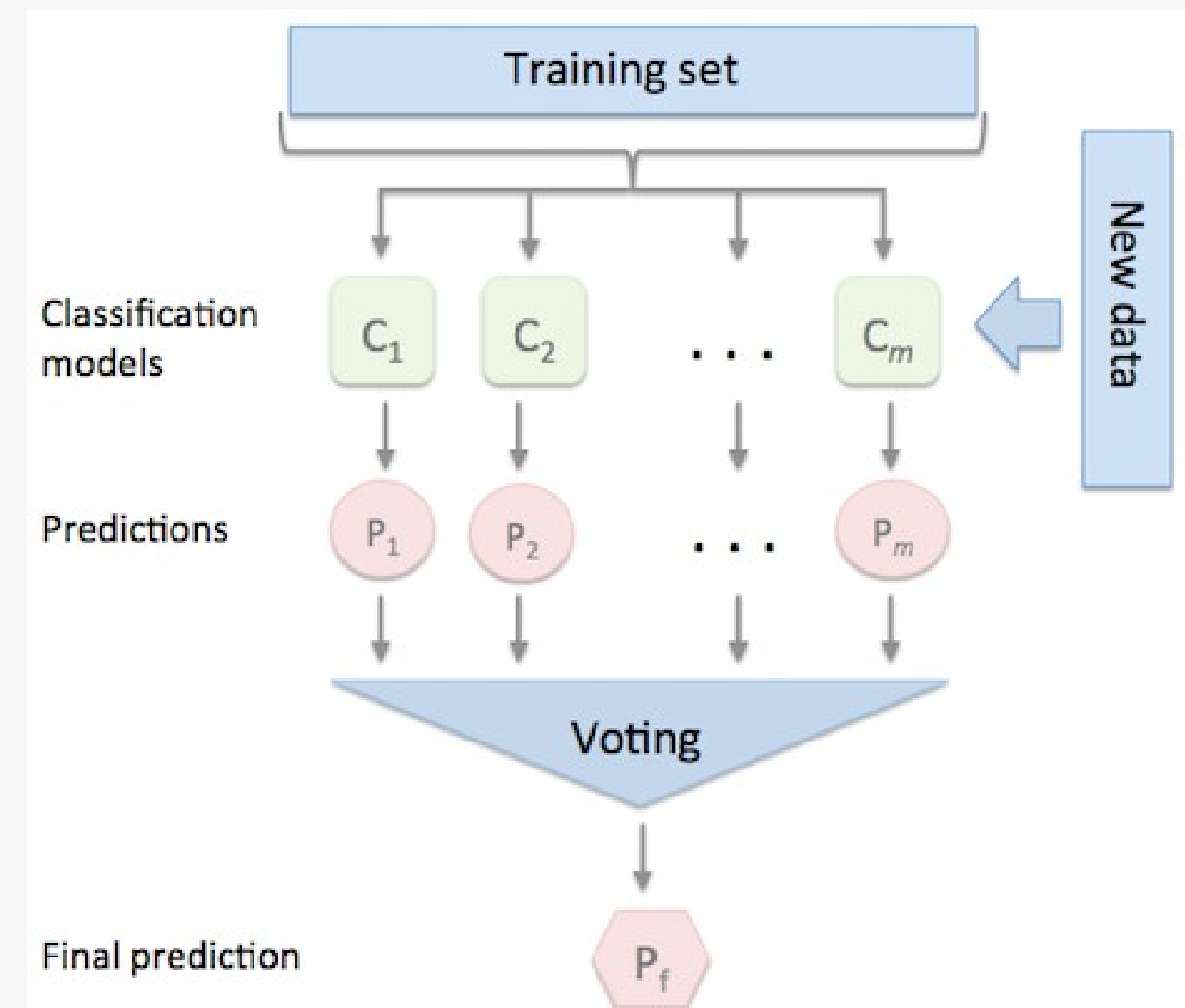
앙상블(Ensemble) 기법 : 여러 개의 분류기를 생성하고 그 예측을 결합함으로써 보다 정확한 예측을 도출하는 기법



머신러닝 종류 - 사전지식

보팅(Voting) : 서로 다른 알고리즘을 가진 분류기 중 투표를 통해 최종 예측 결과를 결정하는 방식

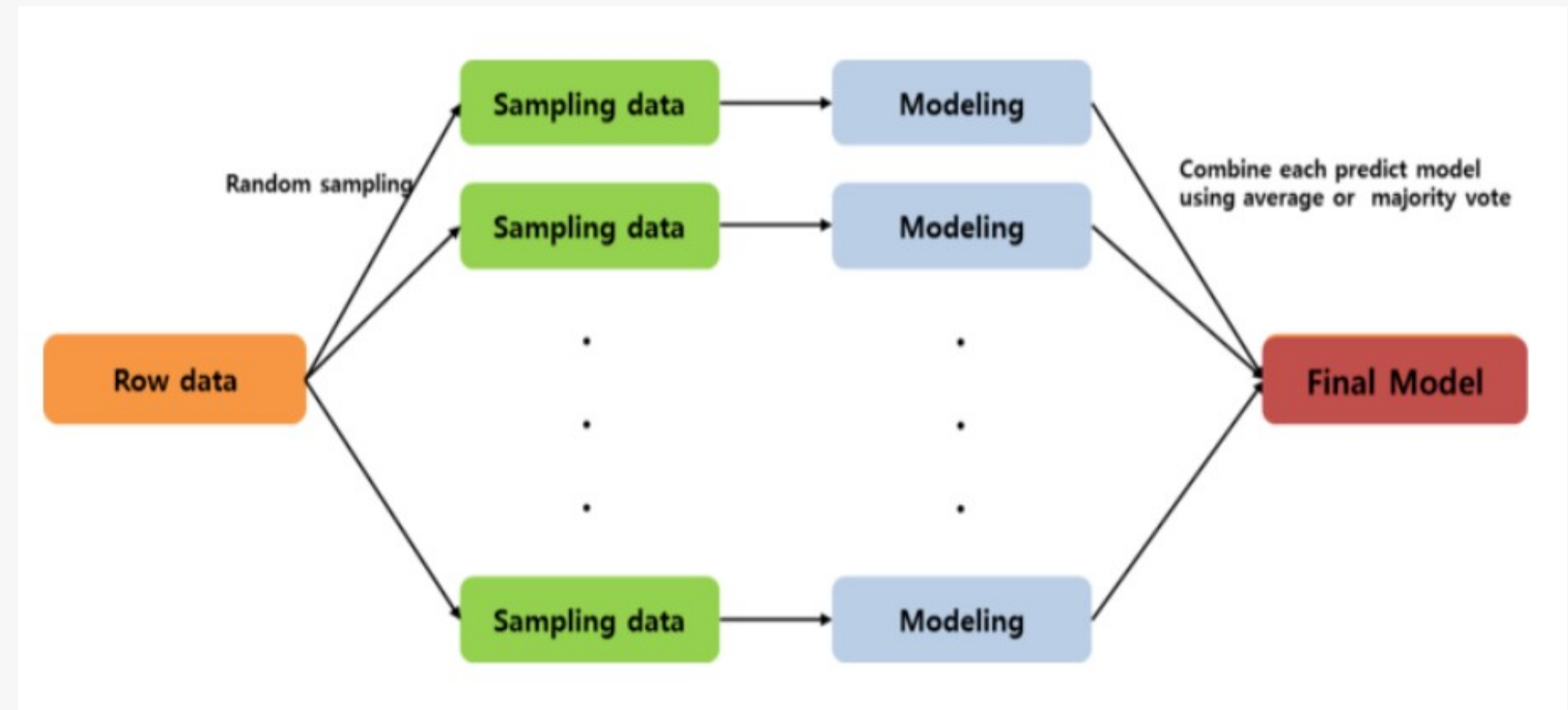
- **하드 보팅(Hard Voting)**
다수결의 원칙을 따르는 방식으로,
각각의 분류기의 결과값 중 가장 많은 걸 따름
- **소프트 보팅(Soft Voting)**
분류기의 확률을 더하고 각각 평균을 내서
확률이 제일 높은 값으로 결과값을 선정
- **보팅을 사용한 알고리즘**
 - VotingClassifier



머신러닝 종류 - 사전지식

배깅(Bagging; Bootstrap aggregating) : 생성된 여러 개의 bootstrap 자료 모델링한 후 결합하여 최종 예측 모델을 산출

- **bootstrap 자료**
 - 원시 데이터(raw)로부터 복원 임의 추출한 크기가 동일한 여러 개의 표본 자료
- 보팅과는 다르게 모두 같은 알고리즘 모델을 사용
 - 과적합 방지에 효과적
- **배깅을 사용한 알고리즘**
 - RandomForestClassifier
 - ExtraTreesClassifier

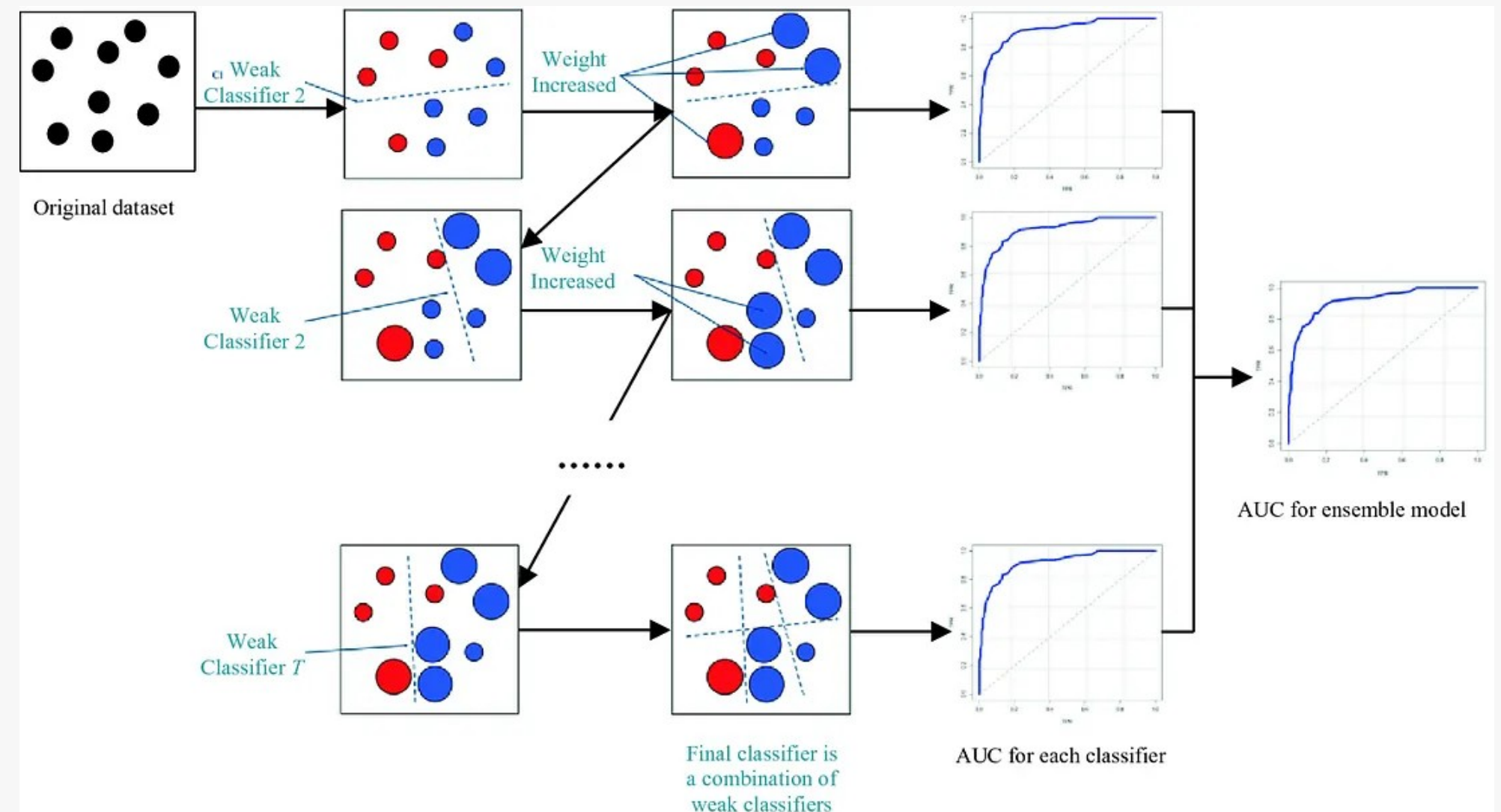


머신러닝 종류 - 사전지식

부스팅(Boosting) : 오분류된 개체들에 집중해 새로운 분류 규칙을 만드는 단계를 반복하는 방법

오분류된 개체들에는 높은 가중치를 부여하고,
정분류된 개체들에는 낮은 가중치를 부여

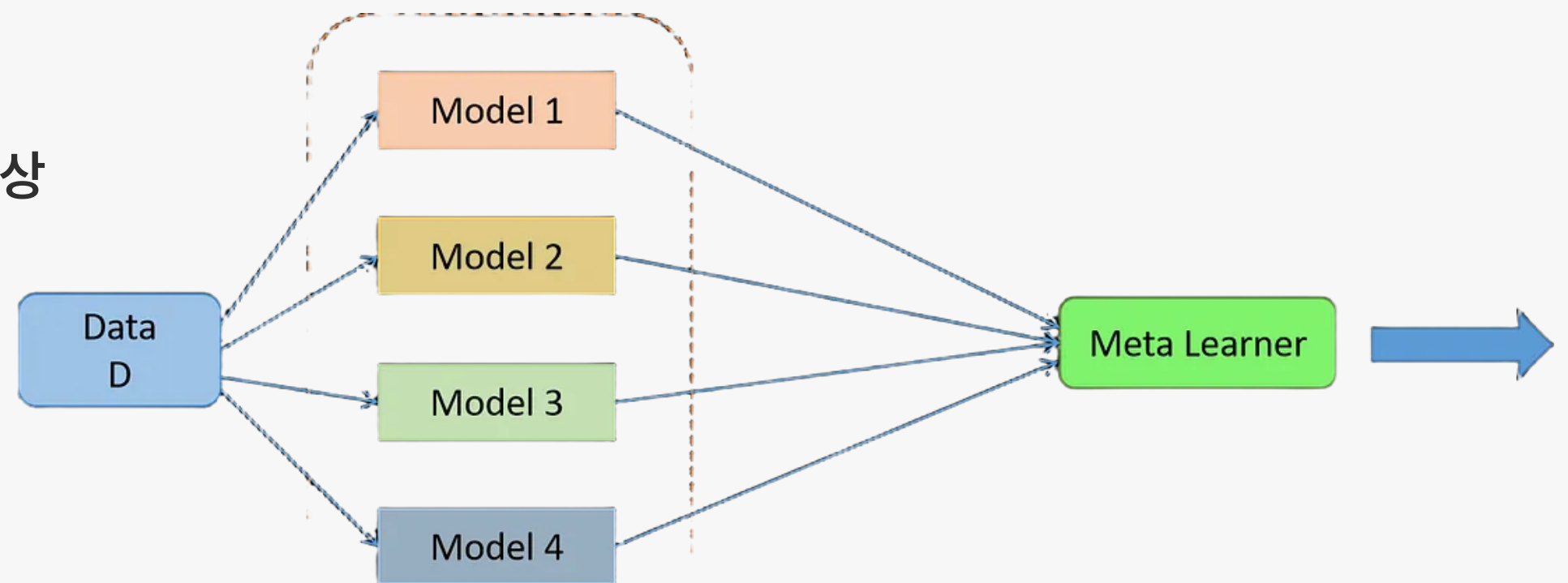
- 높은 정확도를 가지지만,
과대적합의 위험이 크고
직렬적으로 학습해 속도가 느림
- **부스팅을 사용한 알고리즘**
 - AdaBoostClassifier
 - GradientBoostingClassifier
 - XGBoost
 - LightGBM
 - CatBoost



머신러닝 종류 - 사전지식

스태킹(Stacking) : 여러 모델의 예측 결과를 다시 학습 데이터로 사용해 메타모델이 최종 예측을 수행

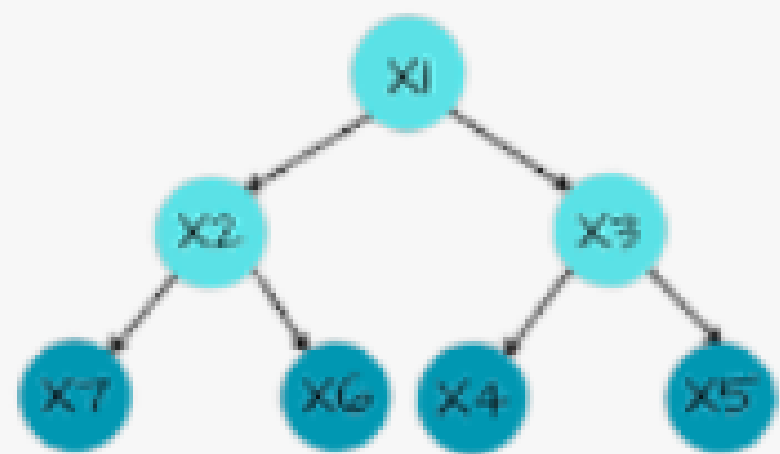
- 구현이 복잡하지만 서로 다른 모델 간 시너지를 낼 수 있어 높은 성능 기대 가능
- 메타 모델을 통한 추가 학습으로 일반화 성능 향상
- 스태킹을 사용한 알고리즘
 - StackingClassifier



머신러닝 종류 - 부스팅 활용 알고리즘 비교

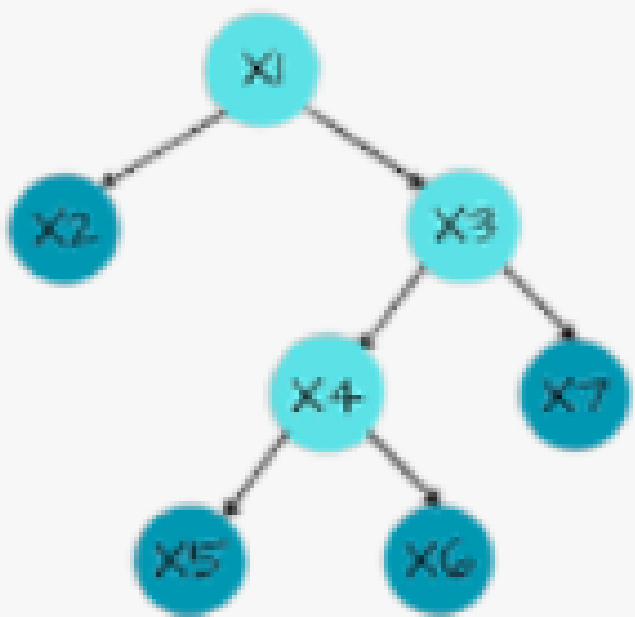
정형 데이터(Tabular Data) 분석에서 가장 널리 쓰이는 대표적인 **그래디언트 부스팅(Gradient Boosting)** 알고리즘

데이터가 매우 **크고**
빠른 학습 속도가 필요할 때

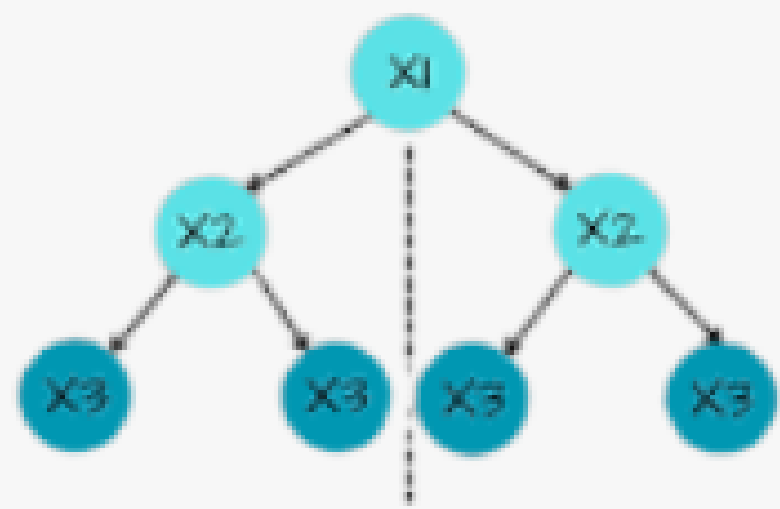


XGBoost

데이터 규모가 적당하고,
안정성과 성능의 균형이 중요할 때



LightGBM



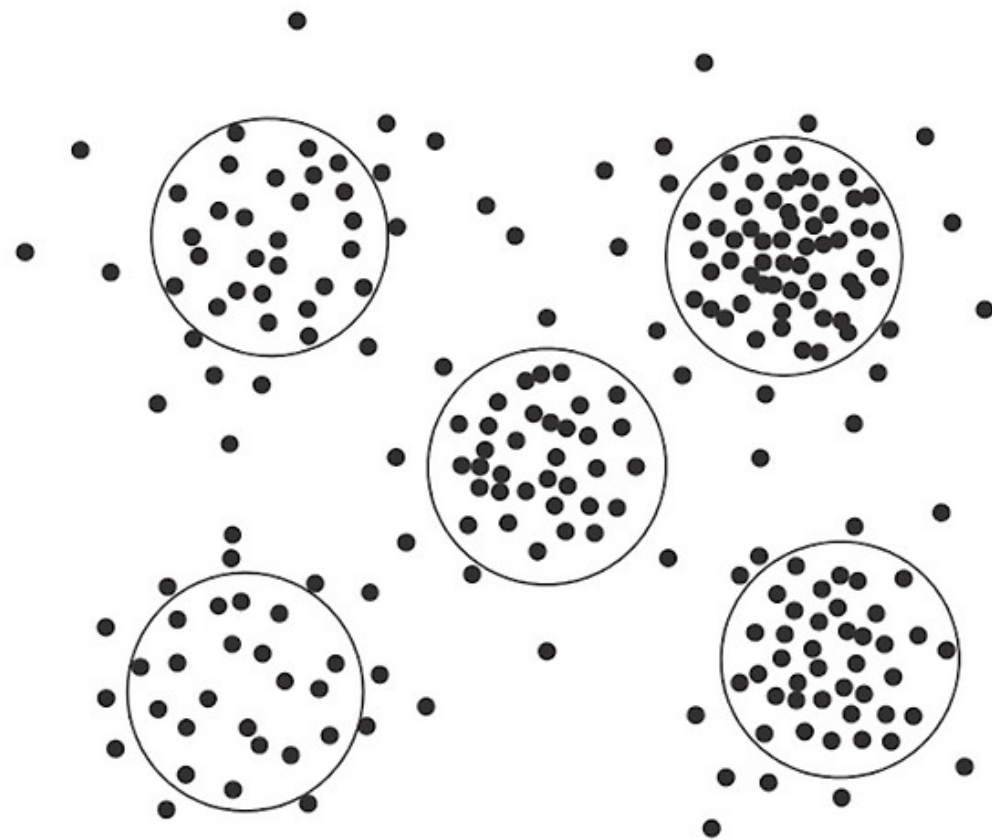
CatBoost

범주형 변수가 많고
전처리를 **최소화**하고 싶을 때

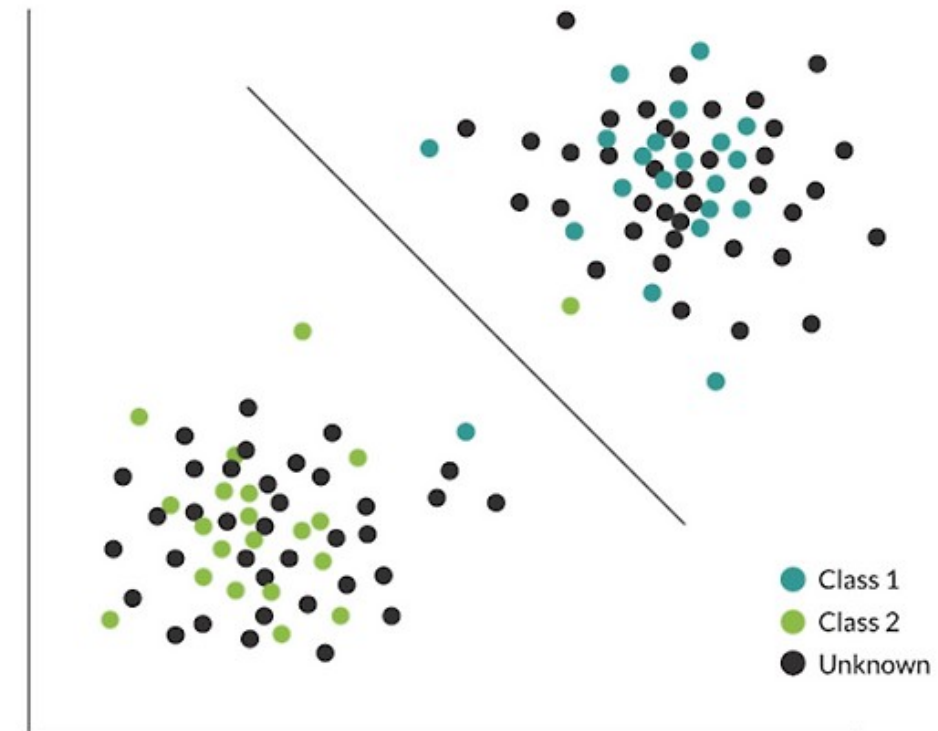
머신러닝 종류

비지도학습 (Unsupervised Learning) : 정답 없이 데이터 내부의 패턴이나 군집을 스스로 발견

Unsupervised

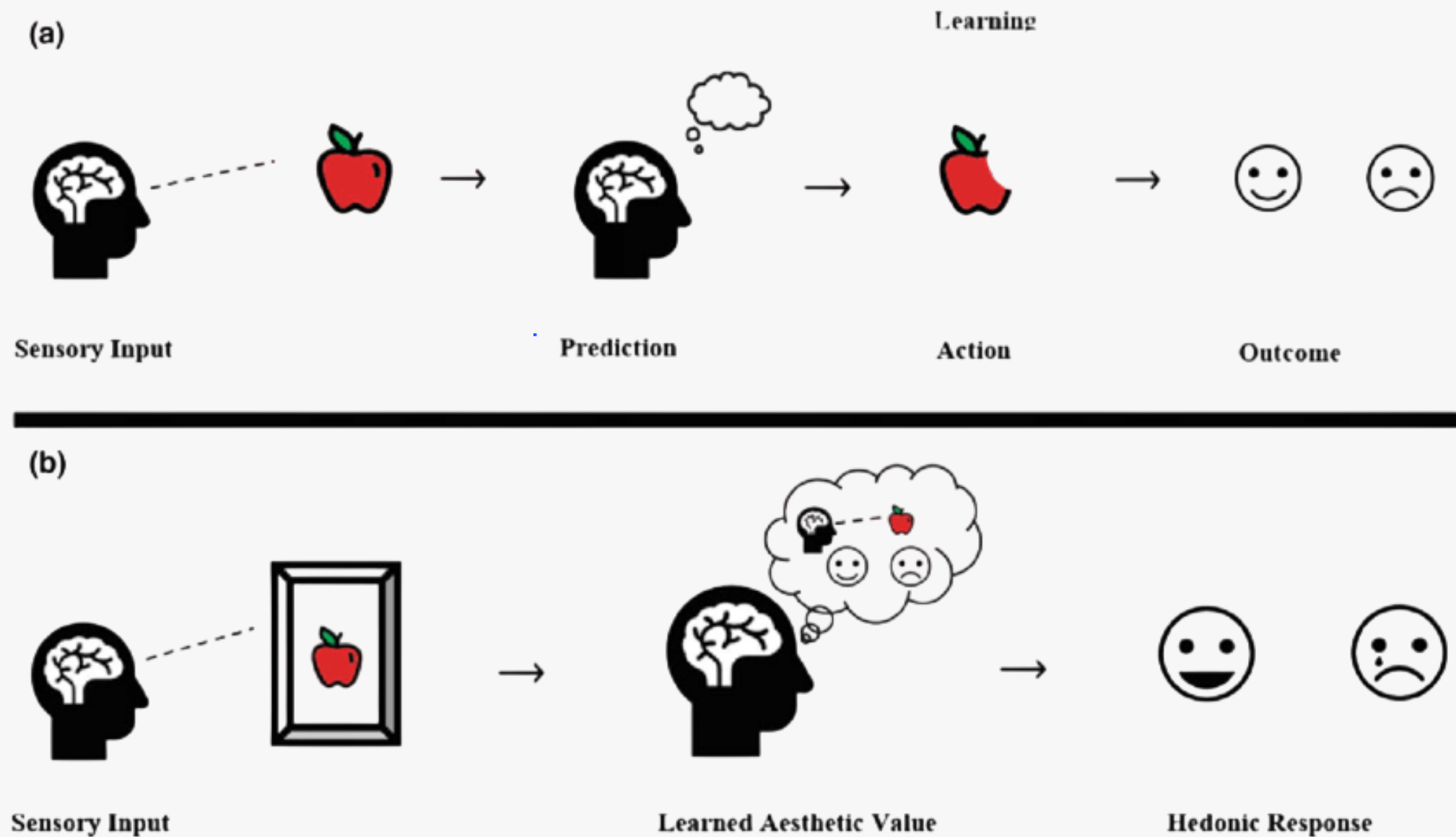


Supervised



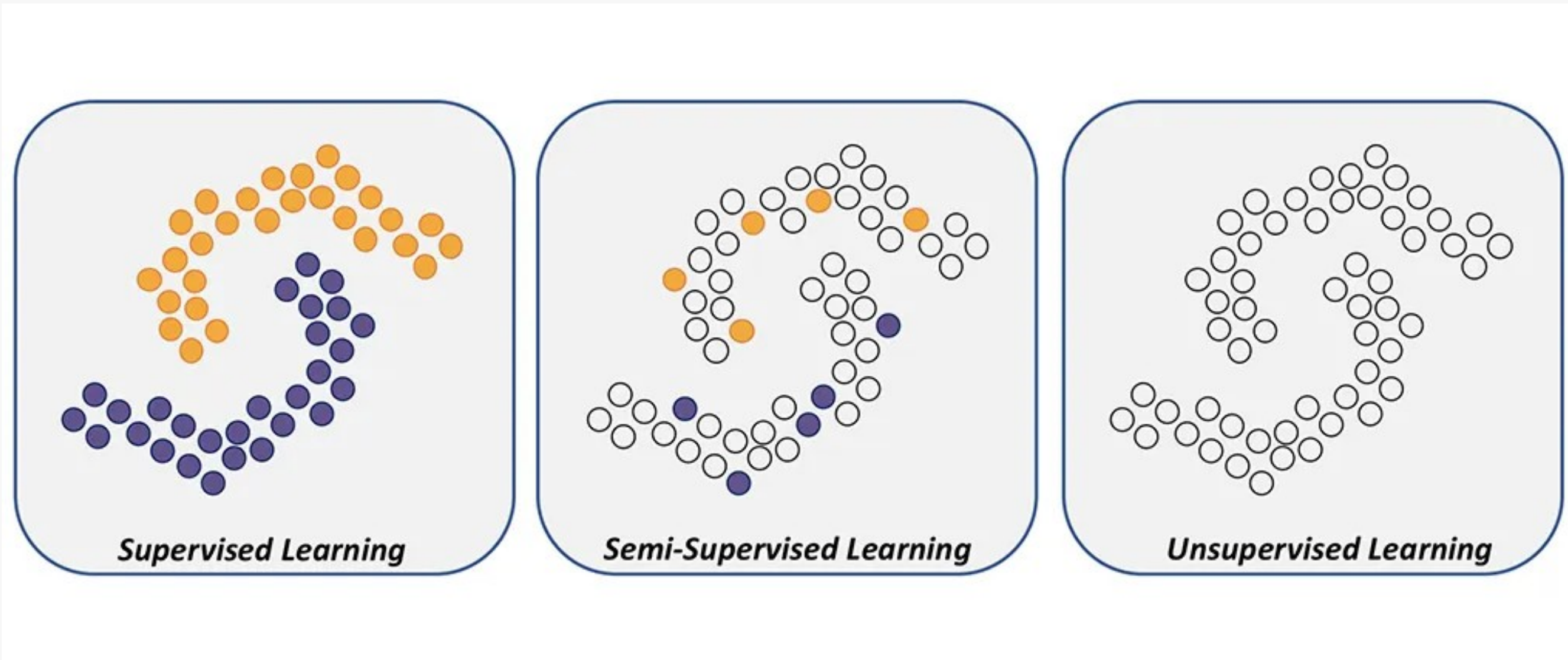
머신러닝 종류

강화학습 (Reinforcement Learning) : 시행착오를 통해 보상을 최대화하는 최적의 행동 방식을 학습



머신러닝 종류

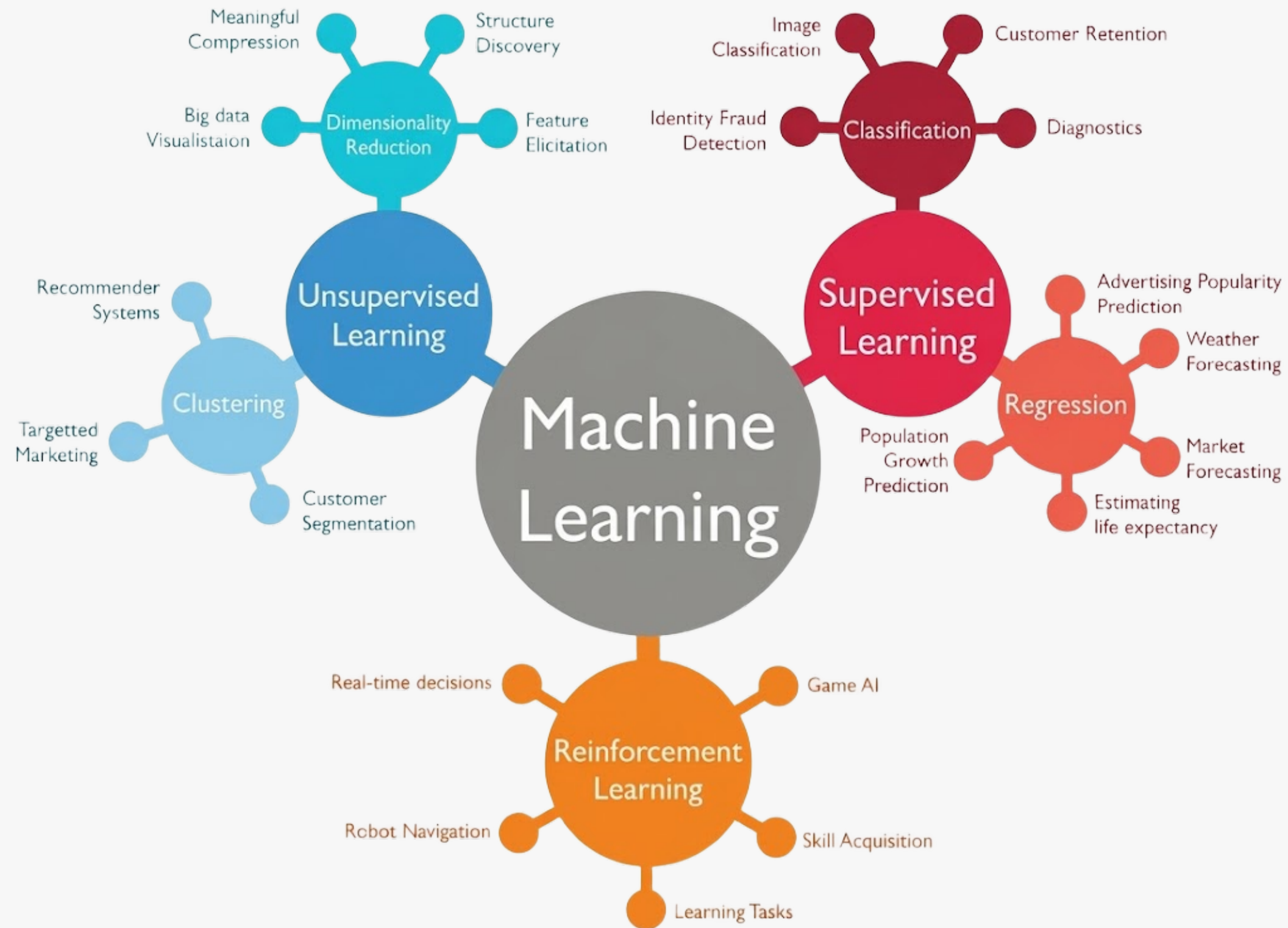
준지도학습 (Semi-supervised Learning) : 소수의 정답 데이터와 다수의 정답 없는 데이터를 함께 활용하는 학습



머신러닝 알고리즘 선택

Data Science Team

Machine Learning Bubble Chart



감사합니다.